

COURSE NOTES

Claude Code Skills

學習筆記

6 個模組，涵蓋 Claude Code Skills 的建立、設定、分享與除錯

資料來源：整理自 [Claude Youtube 官方頻道課程 Claude Code Skills](#)

目錄 Contents

- 01** **What Are Skills?** 什麼是 Skills ?
 - 02** **Troubleshooting Skills** Skills 的除錯與排查
 - 03** **Sharing Skills** 分享與部署 Skills
 - 04** **Skills vs Other Features** Skills 與其他功能的差異
 - 05** **Configuration & Multi-file Skills** 進階設定與多檔案架構
 - 06** **Creating Your First Skill** 建立你的第一個 Skill
-

What Are Skills?

什麼是 Skills？

本節概念：Skill Definition · Storage Locations · skill.md · vs CLAUDE.md

▶ [前往 YouTube 觀看影片](#)

本節重點 KEY TAKEAWAYS

- Skills 是一種 Markdown 檔案，用來一次寫好、讓 Claude 自動在對應時機套用的知識，解決「每次對話都要重複說明」的問題。
- Skills 存放在三個位置：個人（`~/.claude/skills/`）、專案（`./.claude/skills/`）、Plugin（Marketplace 安裝）。
- 每個 Skill 是一個目錄，目錄裡唯一必要的檔案是 `skill.md`。
- Skill 的 frontmatter 包含 `name`（識別用）和 `description`（Claude 用來決定何時啟動）。
- Skills vs CLAUDE.md**：CLAUDE.md 每次對話都載入；Skills 只在請求匹配時按需載入。

核心定義 CORE DEFINITION

每次你向 Claude 解釋你的 coding 規範、PR review 格式、commit 訊息偏好——你都在重複一件事。

Skills 把這些知識寫一次，Claude 按情境自動啟用。

如果你發現自己重複向 Claude 解釋同樣的事情 → 那就是一個 Skill 等著被寫。

三個存放位置 STORAGE LOCATIONS

位置	路徑	適用對象
個人 Skills	<code>~/.claude/skills/</code>	個人偏好、跟著你走，所有專案都能用
專案 Skills	<code>./.claude/skills/</code> （repo 根目錄）	團隊標準，clone repo 就自動擁有
Plugin Skills	透過 Marketplace 安裝	社群分享的通用功能

SKILL 的結構 SKILL STRUCTURE

```
~/.claude/skills/  
├── pr-review/      ← 目錄名稱 = skill name  
    └── skill.md    ← 唯一必要檔案（大寫 SKILL 小寫 .md）
```

`skill.md` 的格式：

```
---
name: pr-review
description: |
  Helps write structured PR descriptions following team template.
  Use when asked to "write a PR description", "review my changes",
  "help me document this PR".
---
```

(以下是 Claude 執行時讀取的指令內容)

SKILLS VS CLAUDE.MD 對比

	CLAUDE.md	Skills
載入時機	每次對話都載入	請求語義匹配才按需載入
適用情境	永遠有效的專案規範	特定任務才需要的知識
範例	「永遠用 TypeScript strict mode」	PR review checklist
Context 佔用	每次都佔	只有觸發時才佔

最佳實踐

Task-specific 的詳細指令放 Skills，避免每次對話都佔用 context window。

● 實作練習 TRY IT NOW

- 1 想一件你最常重複向 Claude 說明的事 (commit 格式、程式碼風格、某个工作流)
- 2 在 `~/.claude/skills/` 建立一個目錄，命名為你的 skill 名稱
- 3 建立 `skill.md`，寫好 `name` 和 `description`，再加上你的指令
- 4 重啟 Claude Code，確認 Skill 出現在 `/list-skills` 中

我的筆記 MY NOTES

Troubleshooting Skills

Skills 的除錯與排查

本節概念：Trigger Matching · File Structure · Priority Conflicts · Runtime Errors

▶ [前往 YouTube 觀看影片](#)

本節重點 KEY TAKEAWAYS

- Skills 不觸發的原因幾乎都是 **description 語義重疊不足**，加入 trigger phrases 可解決。
- `skill.md` 必須放在以 skill 名稱命名的子目錄中，不能直接放在 skills 根目錄。
- 同名 Skill 有 Priority 規則：Enterprise > Personal > Project > Plugin。
- 跑 `claude --debug` 可以看載入錯誤，是最直接的診斷工具。
- Skill 載入但執行失敗 → 通常是外部套件未安裝或腳本缺少執行權限。

四類常見問題 COMMON ISSUES

問題 1：Skill 沒有觸發 (Not Triggering)

Claude 使用**語義匹配**決定啟動哪個 Skill。請求與 description 的語義重疊不夠 → 不觸發。

解法

- 在 description 加入使用者**實際會說的句子** (trigger phrases)
- 測試多種說法：「幫我 profile」「為什麼這麼慢」「讓這個更快」
- 任何說法失敗就把它加進 description

問題 2：Skill 沒出現在清單 (Not Loading)

結構錯誤是最常見原因：

❌ 錯誤：直接放在 skills 根目錄

~/claude/skills/skill.md

✅ 正確：放在以 skill 名稱命名的子目錄

~/claude/skills/pr-review/skill.md

必須確認

檔名必須是 `skill.md` (全小寫)

確認結構正確後還有問題 → 跑 `claude --debug` 查看載入錯誤訊息。

問題 3：用到錯誤的 Skill（Wrong Skill / Conflict）

兩個 Skill 的 description 太相似，或個人 Skill 被企業 Skill 覆蓋。

1

Enterprise
管理設定，最高優先權，永遠覆蓋其他同名 Skill

2

Personal
~/.claude/skills/

3

Project
./.claude/skills/

4

Plugins
最低優先權

若個人 Skill 與企業 Skill 同名 → **企業版永遠優先**。解法：把個人 Skill 改成更具體的名稱。

問題 4：執行失敗（Runtime Failure）

- 外部套件需先安裝，並在 description 中說明
- 腳本需要執行權限：`chmod +x script.sh`（Windows 也用 `/` 路徑分隔符）

快速排查清單 QUICK CHECKLIST

症狀	解法
沒觸發	優化 description，加入 trigger phrases
沒載入	確認路徑結構、 <code>skill.md</code> 檔名、YAML 語法
用到錯的 Skill	讓 descriptions 更明確，避免語義重疊
被覆蓋	確認 priority 規則，必要時改名
Plugin 不見	清 cache、重新安裝
執行失敗	確認相依套件、檔案執行權限

● 實作練習 TRY IT NOW

1

故意用「錯誤的」路徑放一個 `skill.md`，確認它不出現在清單中

2

修正路徑後重啟，確認 Skill 出現

3

修改 description 加入兩個你會說的 trigger phrases，測試它們是否都能觸發

4

用 `claude --debug` 啟動，觀察 Skill 載入的 log 訊息



 我的筆記 MY NOTES

Sharing Skills

分享與部署 Skills

本節概念：Repository Sharing · Plugins · Enterprise Deployment · Sub-agents

► [前往 YouTube 觀看影片](#)

本節重點 KEY TAKEAWAYS

1. 最簡單的分享方式：把 Skills 提交到 repo 的 `.claude/skills/`，clone 就自動擁有。
2. Plugin 是跨 repository 分發 Skills 的方式，上架 Marketplace 後其他人可下載安裝。
3. 企業管理員可透過 Managed Settings 組織級部署，優先權最高。
4. **Sub-agents 不自動繼承 Skills**，必須在 agent.md 的 `skills` 欄位明確列出。
5. 內建 agents（Explorer、Plan、Verify）**完全無法存取** Skills，只有自訂 sub-agent 可以。

三種分享方式 SHARING METHODS

方式 1：提交到 Repository（最簡單）

```
.claude/  
└─ skills/  
    └─ your-skill/  
        └─ skill.md
```

- clone repo 自動擁有，push 更新後大家 pull 即可
- 適合：團隊 coding 規範、專案特定 workflow、需要 reference codebase 的 Skill

方式 2：透過 Plugin 分發

```
your-plugin/  
└─ skills/  
    └─ your-skill/  
        └─ skill.md
```

- 在 plugin 專案建立 `skills/` 目錄，結構與 `.claude/skills/` 相同
- 上架 Marketplace 後其他使用者可下載安裝
- 適合：不限特定專案的通用 Skill、開源社群分享

方式 3：Enterprise 管理員部署

- 透過 Managed Settings 組織級部署
- **Priority 最高**，會覆蓋所有同名的 Personal / Project / Plugin Skill

- 適合：強制標準、合規需求（must 關鍵字）、安全流程

SUB-AGENTS 與 SKILLS SUB-AGENTS & SKILLS

重要：SUB-AGENTS 不自動繼承 SKILLS

Sub-agent 啟動時是全新的乾淨 context。你的 Skills **不會** 自動被 sub-agent 看到。

Agent 類型	能用 Skills 嗎？
內建 agents（Explorer、Plan、Verify）	❌ 完全無法存取
自訂 sub-agent	✅ 可以，但要明確列出

作法：在 agent.md 加入 skills 欄位

```
.claude/agents/frontend-reviewer.md:

---
name: frontend-reviewer
description: Reviews frontend PRs for style and accessibility
skills:
  - pr-review
  - brand-guidelines
---
```

（以下是 agent 的指令）

注意：Sub-agent 的 Skills 是**啟動時一次全部載入**，不是按需載入。只列永遠相關的 Skills，避免浪費 context。

使用情境

「不同 sub-agent 需要不同的 Skills」—— 例如 frontend-reviewer 用 UI 規範，backend-reviewer 用 API 規範。

● 實作練習 TRY IT NOW

- 1 把你的個人 Skill 複製一份到目前專案的 `.claude/skills/`，確認它能在專案中使用
- 2 建立一個 `.claude/agents/` 目錄，嘗試寫一個自訂 sub-agent 並指定一個 Skill
- 3 思考：你的團隊裡有哪些「每個人都要遵守」的規範適合做成 Skill 提交到 repo？



 我的筆記 MY NOTES

Skills vs Other Claude Code Features

Skills 與其他功能的差異

本節概念：CLAUDE.md · Sub-agents · Hooks · MCP Servers · Feature Selection

▶ [前往 YouTube 觀看影片](#)

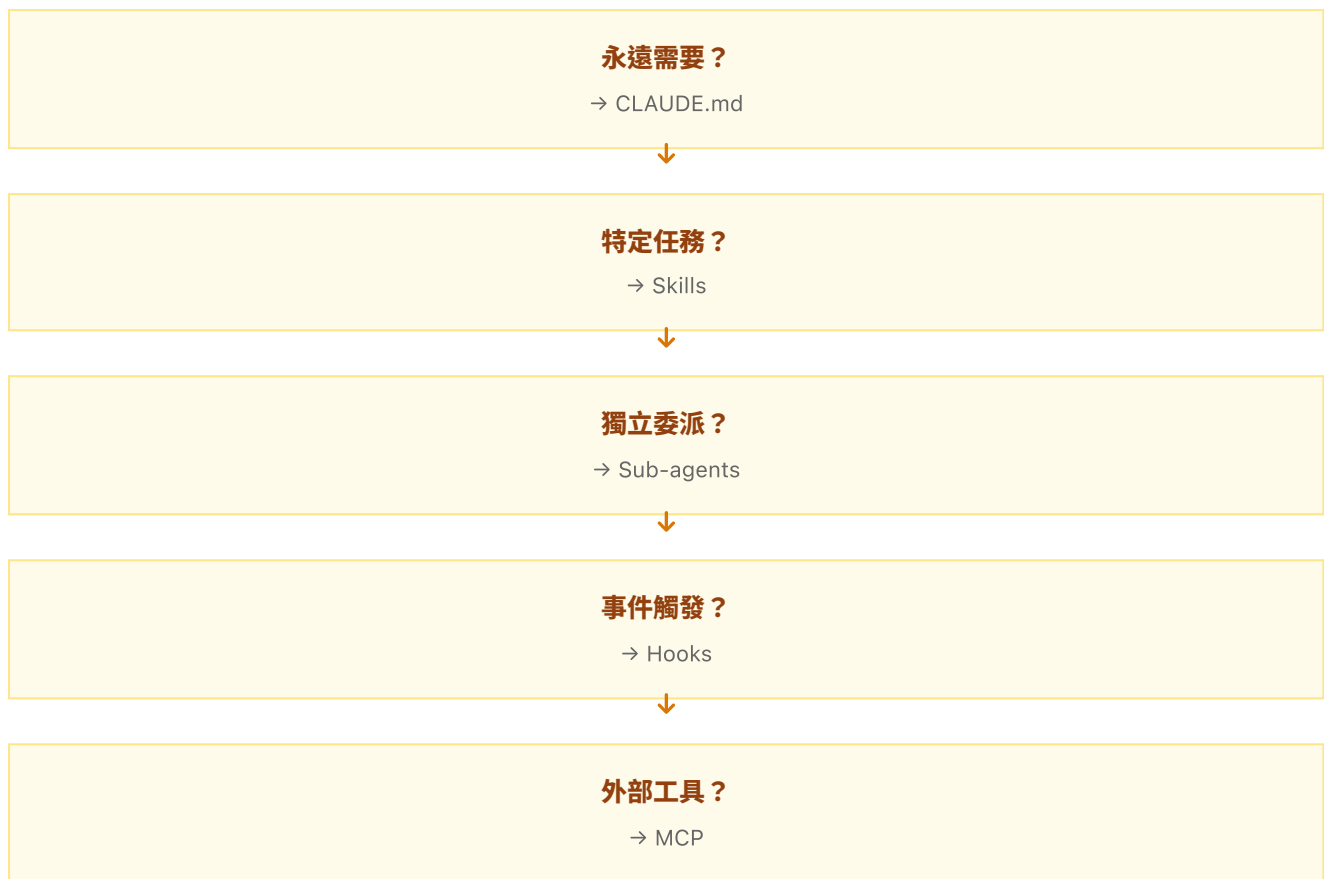
本節重點 KEY TAKEAWAYS

- 1. Claude Code 有五種客製化工具，各自解決不同問題，不能互相取代。
- 2. **Skills vs CLAUDE.md**：CLAUDE.md 永遠載入，Skills 按需載入；詳細流程說明放 Skills 而非 CLAUDE.md。
- 3. **Skills vs Sub-agents**：Skills 增強當前對話的知識；Sub-agents 在獨立 context 執行任務。
- 4. **Skills vs Hooks**：Skills 是請求驅動（你說什麼）；Hooks 是事件驅動（Claude 做了什麼）。
- 5. 大多數工作流會同時用多個工具，它們可以互補組合。

五種工具對比 FEATURE COMPARISON

工具	觸發時機	執行位置	適用情境
CLAUDE.md	每次對話都載入	主對話 context	永遠適用的規範、框架偏好、全域設定
Skills	請求語義匹配才載入	主對話 context	特定任務的專業知識、有時才需要的指令
Sub-agents	被委派時啟動	獨立 context	獨立任務委派、需要隔離的工作、不同工具存取
Hooks	事件觸發（存檔、工具呼叫）	系統層	每次存檔 lint、工具呼叫前驗證、自動化副作用
MCP Servers	工具呼叫時	外部服務	連接外部 API、資料庫、第三方工具

決策框架 DECISION FRAMEWORK



用 CLAUDE.md 當：

- 規則永遠需要套用（「永遠不要修改 DB schema」）
- 框架偏好、程式碼風格、語言設定

用 Skills 當：

- 知識只在特定任務時需要
- 詳細流程說明（放進 CLAUDE.md 會讓每次對話都很肥）

用 Sub-agents 當：

- 想委派任務到獨立執行環境（不污染主 context）
- 需要與主對話不同的工具存取權

用 Hooks 當：

- 每次存檔都要跑的操作（自動 lint）
- 特定工具呼叫前的驗證（不是「Claude 決定」，而是「事件觸發」）

典型組合 TYPICAL SETUP

CLAUDE.md	→ TypeScript strict mode、永遠不改 DB schema
Skills	→ PR review checklist、brand guidelines
Hooks	→ 存檔時自動跑 ESLint
Sub-agents	→ 委派給 frontend-reviewer / backend-reviewer

原則

不要把所有東西都塞進 Skills。正確工具用在正確地方，是客製化 Claude Code 最重要的判斷力。

● 實作練習 TRY IT NOW

- 1 列出你目前在 CLAUDE.md 裡的規則，哪些是「特定任務才需要的」？把它們移到 Skills
- 2 想一個你希望「每次存檔自動執行」的操作 → 那是 Hook，不是 Skill
- 3 規劃你的 Claude Code 客製化架構：CLAUDE.md 放什麼、Skills 放什麼、Hooks 放什麼

我的筆記 MY NOTES

Configuration & Multi-file Skills

進階設定與多檔案架構

本節概念：Metadata Fields · allowed_tools · Progressive Disclosure · Scripts

▶ [前往 YouTube 觀看影片](#)

本節重點 KEY TAKEAWAYS

1. `skill.md` frontmatter 除了 name 和 description，還有 `allowed_tools`（限制工具）和 `model`（指定模型）。
2. **Progressive Disclosure**：主指令放 `skill.md`（500 行以內），詳細參考資料放外部檔案，按需讀取。
3. Scripts 可以執行而不讀入 context，只消耗 output 的 tokens。
4. Description 要回答兩個問題：這個 Skill 做什麼？何時應該用它？
5. 超過 500 行就考慮拆分，或把部分內容移到 `references/`。

SKILL.MD METADATA 欄位 METADATA FIELDS

```
---
name: pr-review          # 必填：小寫+連字號，max 64 字，須與目錄名稱一致
description: |            # 必填：Claude 用來決定何時啟動，max 1024 字
  Writes structured PR descriptions following our template.
  Use when: "write a PR description", "review my changes",
  "help me document this", "write PR notes".
allowed_tools:           # 選填：限制此 Skill 可用的工具
  - Read
  - Bash
model: claude-sonnet-4-6 # 選填：指定使用哪個 Claude 模型
---
```

寫好 DESCRIPTION 的關鍵 WRITING EFFECTIVE DESCRIPTIONS

Description 是 Claude 決定是否觸發的唯一依據。一個好的 description 要回答：

1. 這個 Skill 做什麼？（What it does）
2. 何時應該用它？（When to use it — trigger phrases）

✗ 太模糊

Helps with dogs.

✓ 清楚明確

Helps with dog breed identification and care recommendations. Use when asked to "identify this breed", "what breed is this?", "tell me about dog breeds", or any dog-related questions.

ALLOWED_TOOLS：限制工具存取

- 列出的工具 = Skill 啟動時 Claude 只能用這些工具（無需詢問權限）
- 省略 `allowed_tools` → 不限制，沿用一般權限模型
- 常用情境：只讀分析型 Skill → 只列 `Read`，避免意外修改

多檔案架構 MULTI-FILE STRUCTURE

當 Skill 越來越複雜，Progressive Disclosure 讓你把「全部塞進一個大檔案」變成「按需讀取」：

```
pr-review/
├─ skill.md           ← 核心指令，保持 < 500 行
├─ references/
│   └─ architecture.md ← 只有被問到系統設計時才讀
├─ scripts/
│   └─ validate.sh     ← 執行腳本（不讀入 context，直接跑）
└─ assets/
    └─ template.json   ← 模板資料、範例等
```

在 `skill.md` 裡這樣引用：

如果被問到系統架構相關問題，讀 `references/architecture.md`。
如需驗證環境，執行 `scripts/validate.sh`（不要讀取它的內容）。

讓 Claude 讀外部檔案 vs. 執行腳本的差異：

- 讀取 → 內容進入 context window，消耗 tokens
- 執行 → 只有 output 進入 context，腳本內容不佔空間

SCRIPTS 的使用時機 WHEN TO USE SCRIPTS

- 環境驗證（確認套件是否安裝）
- 需要一致性的資料轉換
- 比 LLM 生成的程式碼更可靠的操作

告訴 Claude 「執行」這個 script，而不是「讀取」它。

● 實作練習 TRY IT NOW

- 1 為你現有的 Skill 加入 `allowed_tools` ，限制它只能用 `Read` ，測試是否生效
- 2 建立一個 `references/` 子目錄，把過長的參考資料移進去，在 `skill.md` 中用條件式引用
- 3 寫一個簡單的 validation script，讓 Skill 執行它來確認環境是否就緒
- 4 確認你的 `skill.md` 是否超過 500 行，如果是，規劃如何拆分

■ 我的筆記 MY NOTES

Creating Your First Skill

建立你的第一個 Skill

本節概念：Skill Creation · Loading Flow · Priority Rules · Update & Delete

► [前往 YouTube 觀看影片](#)

本節重點 KEY TAKEAWAYS

1. 建立 Skill = 在 skills 目錄下建立一個以 skill 名稱命名的目錄 + 一個 `skill.md` 檔案。
2. Claude Code 啟動時只載入 name + description，**不載入全文**，直到被觸發才讀完整內容。
3. 觸發時 Claude 會先詢問確認是否載入，讓你知道它在用哪個 Skill。
4. Priority：Enterprise > Personal > Project > Plugin。
5. 更新 Skill → 直接編輯 `skill.md`，重啟 Claude Code；刪除 → 刪整個目錄，重啟。

建立流程 STEP-BY-STEP

```
# 1. 建立目錄 (個人 Skill 範例)
mkdir -p ~/.claude/skills/explain-code

# 2. 建立 skill.md
# (寫好 frontmatter + 指令)

# 3. 重啟 Claude Code
# (Skill 在啟動時載入，修改後需重啟)

# 4. 驗證 Skill 已載入
# 在對話中輸入：/list-skills 或直接問 "what skills are available"
```

skill.md 範例

```
---
name: explain-code
description: |
  Explains code using visual diagrams and analogies.
  Use when asked to "explain this function", "help me understand this code",
  "what does this do", "walk me through this", "break this down for me".
---
```

當解釋程式碼時，請依以下步驟：

1. 先用一個日常生活的比喻說明核心概念
2. 用流程圖（ASCII 圖或文字描述）展示資料流
3. 逐步說明每個關鍵步驟
4. 最後給一個完整的使用範例

CLAUDE CODE 如何載入 SKILL HOW SKILLS LOAD



為什麼要確認步驟？ 讓你清楚知道 Claude 正在用哪個 Skill，保持透明度。

❑ PRIORITY 規則 PRIORITY RULES

1 Enterprise (最高)
管理設定，無法被個人/專案 Skill 覆蓋

2 Personal
~/.claude/skills/

3 Project
./.claude/skills/

4 Plugins (最低)
Marketplace 安裝的 Skills

命名建議

避免泛用名稱：review → 容易衝突

用描述性名稱：frontend-pr-review、security-audit → 明確、不易衝突

❑ 更新與刪除 UPDATE & DELETE

操作	作法
更新 Skill	直接編輯 skill.md → 重啟 Claude Code
刪除 Skill	刪除整個 skill 目錄 → 重啟 Claude Code

● 最終挑戰 FINAL CHALLENGE

- 1 建立一個解決你實際痛點的 Skill (從你最常重複說的事情出發)
- 2 寫好 description，包含至少 3 個 trigger phrases
- 3 重啟後測試，確認觸發成功
- 4 把它提交到你的 repo，讓團隊成員也能使用
- 5 思考：這個 Skill 未來需要哪些 references 或 scripts？先規劃好目錄結構



我的筆記 MY NOTES

資料來源：整理自 [Claude Youtube 官方頻道課程](#) [Claude Code Skills](#)